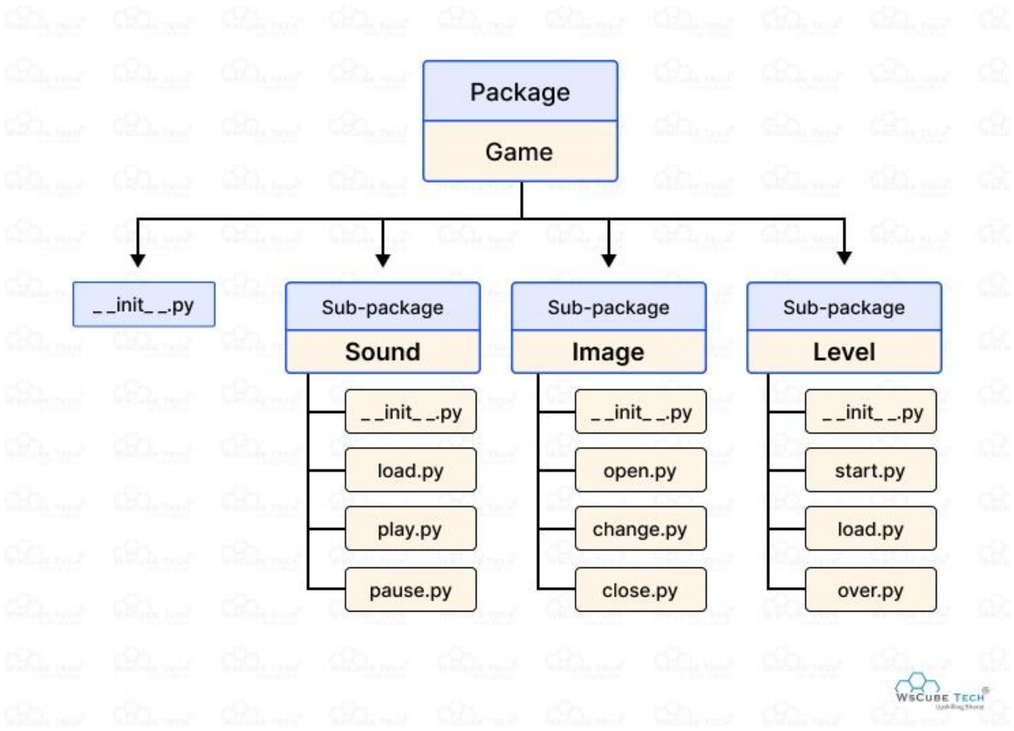
Welcome & Module Induction

- **What we learned previously:** In Module 14, we built an automated "Safety Net" using testing and mocking to ensure our code is bug-free.
- **What we are learning today:** We are going global. We will learn how to structure our code into a professional library, write its "Identity Card" (pyproject.toml), and publish it to the **Python Package Index (PyPI)**.
- **Why this will be helpful:** Think about the tools you use every day: numpy, pandas, torch, fastapi. These weren't just scripts; they were **packaged**. If you want your project to be used by other students or companies, you must provide it as an installable package.

Library Structuring (The Blueprint)

- **The Problem:** A single script is hard to share. If you have 5 files and 10 dependencies, your user will get lost trying to set it up.
- **The Solution:** A standardized directory structure. We group our logic into a "Source" folder (src/) and separate our tests, documentation, and metadata.
- **The __init__.py file:** This tiny, often empty file is the "Magic Key." It tells Python that a folder is not just a collection of files, but a **Package** that can be imported.



pyproject.toml (The ID Card)

- **The Modern Standard:** In the past, developers used setup.py. Today, the world has moved to pyproject.toml.
- **The Contents:** This file contains everything the world needs to know about your library:
- **Metadata:** Name, Version (e.g., 0.1.0), Author (Zaryab Ahmad), and License.
- **Dependencies:** What other libraries (like requests or numpy) does your code need to run?
- **Build System:** Which tool should be used to compile your code (e.g., setuptools or poetry)?

```toml
[project]
name = "mypackage"
version = "2023.05.1"
description = "A package for demonstrating Python packaging"

[build-system]
requires = ["setuptools >= 61.0.0"]
build-backend = "setuptools.build_meta"

[tool.setuptools.packages.find]
where = ["src"]
```

PyPI & Twine (The Delivery Truck)

- **PyPI:** The "Python Package Index." This is the central warehouse where all Python software lives.
- **Twine:** This is the secure "Delivery Truck" that carries your packaged code from your laptop to the PyPI warehouse.
- **The Result:** Once uploaded, any person on Earth can open a terminal and type pip install your-package-name to use your work.